Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

По дисциплине «АПО ЭВМ»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – разработка ассемблерной вставки»

Выполнил:
Студент 4 курса
Группы АС-66
Лысюк Р. А.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками.

Ход работы

1. Изучить следующие теоретические сведения: программная модель микропроцессора; регистры и их назначение; организация памяти; основные конструкции и базовые команды ассемблера (используя материалы лекций и методические указания).
2. Написать ассемблерную вставку, реализующую обработку строки согласно варианту, выданному преподавателем. Оформить ее в виде отдельной функции.
3. Реализовать данную обработку строки также в виде функции на C++. Сравнить быстродействие обоих вариантов. В отчете отразить выводы.

Содержание отчета:
- титульный лист;
- описание варианта задания;
- текст программы;

Вариант 4: Даны 2 строки. Совместить четные символы одной строки с нечетными другой.

Текст программы:

```
#include <iostream>
#include <chrono>
#include <cstring>
#include <clocale>

using namespace std;

extern "C" void combineStringsAsm(const char* s1, const char* s2, char* result) {
    __asm {
        push    esi
        push    edi
        push    ebx

        mov     esi, s1
        mov     edi, s2
        mov     ebx, result

        xor ecx, ecx

        loop_s1 :
        mov     al, [esi + ecx]
```

```
        test    al, al
        jz      loop_s2_start

        test    ecx, 1
        jnz     skip_s1

        mov[ebx], al
        inc     ebx

        skip_s1 :
    inc     ecx
        jmp     loop_s1

        loop_s2_start :
    xor ecx, ecx

        loop_s2 :
    mov     al, [edi + ecx]
        test    al, al
        jz      done

        test    ecx, 1
        jz      skip_s2

        mov[ebx], al
        inc     ebx

        skip_s2 :
    inc     ecx
        jmp     loop_s2

        done :
    mov     byte ptr[ebx], 0

        pop     ebx
        pop     edi
        pop     esi
  }
}
```

```
void combineStringsCpp(const char* s1, const char* s2, char* result) {
    int k = 0;

    for (int i = 0; s1[i] != '\0'; ++i) {
        if (i % 2 == 0) {
            result[k++] = s1[i];
        }
    }

    for (int i = 0; s2[i] != '\0'; ++i) {
        if (i % 2 != 0) {
            result[k++] = s2[i];
        }
    }

    result[k] = '\0';
}

int main() {
    setlocale(LC_ALL, "Russian");

    const char* s1 = "ABCDEFGHIJ";
    const char* s2 = "0123456789";

    char resultAsm[256] = { 0 };
    char resultCpp[256] = { 0 };

    combineStringsAsm(s1, s2, resultAsm);
    combineStringsCpp(s1, s2, resultCpp);

    cout << "String 1: " << s1 << endl;
    cout << "String 2: " << s2 << endl;
    cout << "Result (ASM): " << resultAsm << endl;
    cout << "Result (C++): " << resultCpp << endl;

    if (strcmp(resultAsm, resultCpp) == 0) {
        cout << "[OK] Results match!" << endl << endl;
    }
    else {
        cout << "[ERROR] Results differ!" << endl << endl;
```

```
    }

    const int N = 1000000;

    for (int i = 0; i < 10000; ++i) {
        combineStringsAsm(s1, s2, resultAsm);
    }
    for (int i = 0; i < 10000; ++i) {
        combineStringsCpp(s1, s2, resultCpp);
    }

    auto start = chrono::high_resolution_clock::now();
    for (int i = 0; i < N; ++i) {
        combineStringsAsm(s1, s2, resultAsm);
    }
    auto end = chrono::high_resolution_clock::now();
    long long asmNs = chrono::duration_cast<chrono::nanoseconds>(end - start).count();

    start = chrono::high_resolution_clock::now();
    for (int i = 0; i < N; ++i) {
        combineStringsCpp(s1, s2, resultCpp);
    }
    end = chrono::high_resolution_clock::now();
    long long cppNs = chrono::duration_cast<chrono::nanoseconds>(end - start).count();

    cout << "Iterations: " << N << endl;
    cout << "ASM total:  " << asmNs << " ns  (" << asmNs / N << " ns/call)" << endl;
    cout << "C++ total:  " << cppNs << " ns  (" << cppNs / N << " ns/call)" << endl;

    double ratio = (double)cppNs / (double)asmNs;
    cout << "Ratio (C++ / ASM): " << ratio << endl;

    volatile char sink = resultAsm[0] ^ resultCpp[0];
    (void)sink;

    return 0;
}
```

Результат работы программы:

```
String 1: ABCDEFGHIJ
String 2: 0123456789
Result (ASM): ACEGI13579
Result (C++): ACEGI13579
[OK] Results match!

Iterations: 1000000
ASM total:  12438700 ns  (12 ns/call)
C++ total:  7884600 ns  (7 ns/call)
Ratio (C++ / ASM): 0.633877

C:\ПАПКА ЧТОБ ПЕРЕКИДЫВАТЬ ФАЙЛ CPP НА ГИТ\lab1\Release\lab1.exe (процесс 20212) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

Вывод: Реализованы две функции обработки строк (asm + C++), обе дают верный результат. В Debug ассемблер быстрее в ~1.5–1.7 раза, в Release с /O2 C++ обгоняет ассемблер (8 нс против 12 нс) за счёт векторизации. Вывод: ручной ассемблер оправдан только без оптимизации, в Release MSVC генерирует более эффективный код, чем наивная ручная реализация.